COSC204 Computer Systems

Lecture 17: Communication between processes

Zhiyi Huang

School of Computing, University of Otago

Overview

Last lecture:

- Device controllers and device drivers
- Types of I/O devices
- I/O models
- I/O scheduling and buffering
- User interface for I/O devices

This lecture:

- Portable Operating System Interface (POSIX)
- Cooperating processes
- Creating a child process
- Communication between parent and child processes
- Signals and pipes for inter-process communication

FYI: the Portable Operating System Interface (POSIX)

POSIX[1] (Portable Operating System Interface) is a family of standards for maintaining compatibility between operating systems.

POSIX defines the application programming interface (API), along with command line shells and utility interfaces, for software compatibility with variants of Unix and other operating systems.

- The Single UNIX Specification (SUS) is a similar standard.
- Most modern operating systems are POSIX compliant, including Linux, macOS, and FreeBSD.
- Microsoft Windows NT (released in the 90s) provided a basic POSIX subsystem—now ancient history since WSL provides Linux compatibility.

[1] IEEE Std 1003.1, ISO/IEC 9945

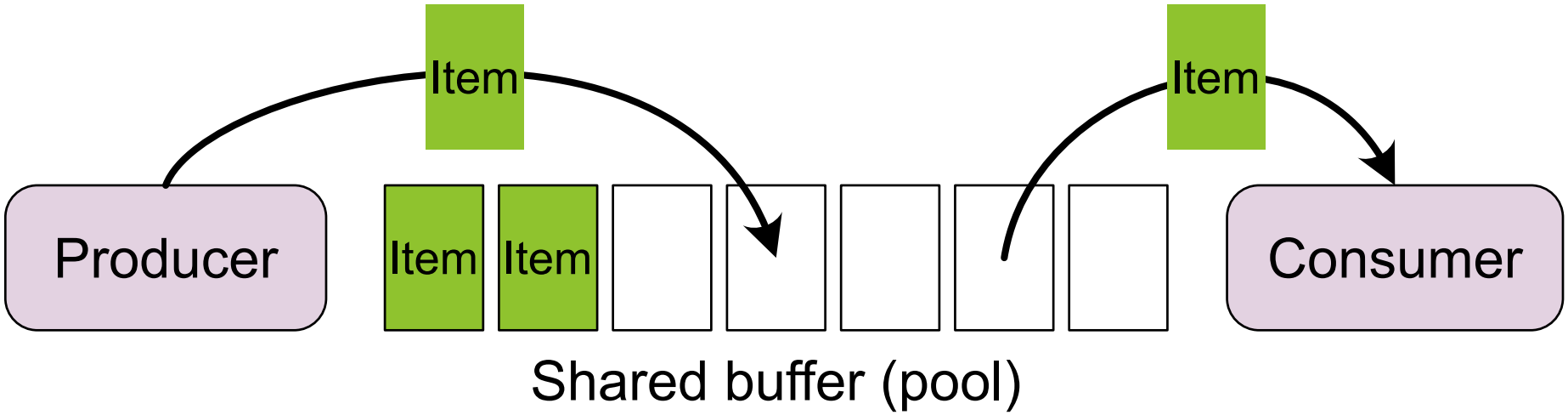
Cooperating processes affect each others' execution

Some terminology:

- Cooperating processes: where the execution of one process can affect the execution of another, e.g., processes used in parallel computing.
- Independent processes: where the execution of one process should have no effect on the execution of another.

Processes that share data will naturally be cooperating processes.

Cooperating processes in producer-consumer model



Coordinating processes in producer-consumer model

- A producer process writes data to a buffer of fixed size.
- A consumer process reads data from the buffer.
- The two processes are scheduled independently by the CPU.

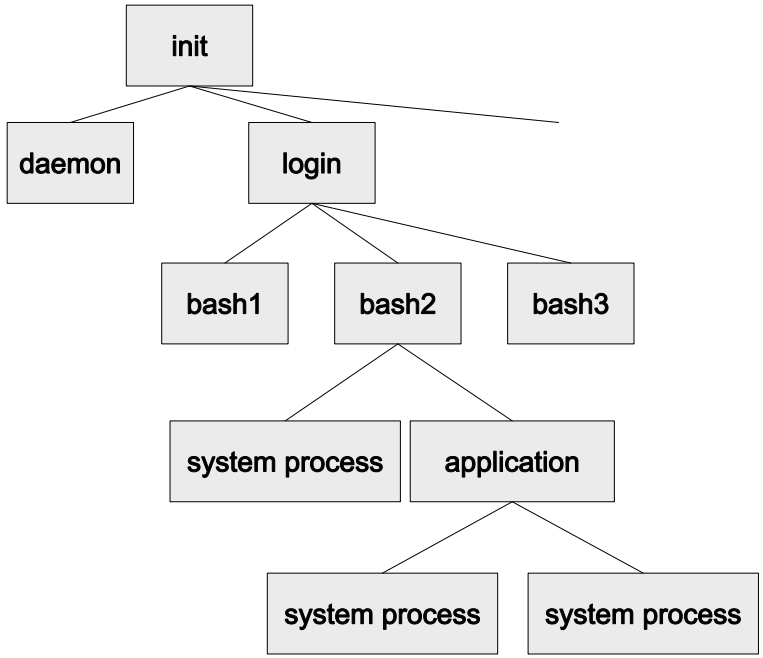
Thus to operate safely:

- the producer process must wait if the buffer becomes full;
- the consumer process must wait if the buffer is empty.

Making this work smoothly involves synchronisation between the two processes. We will return to this ...

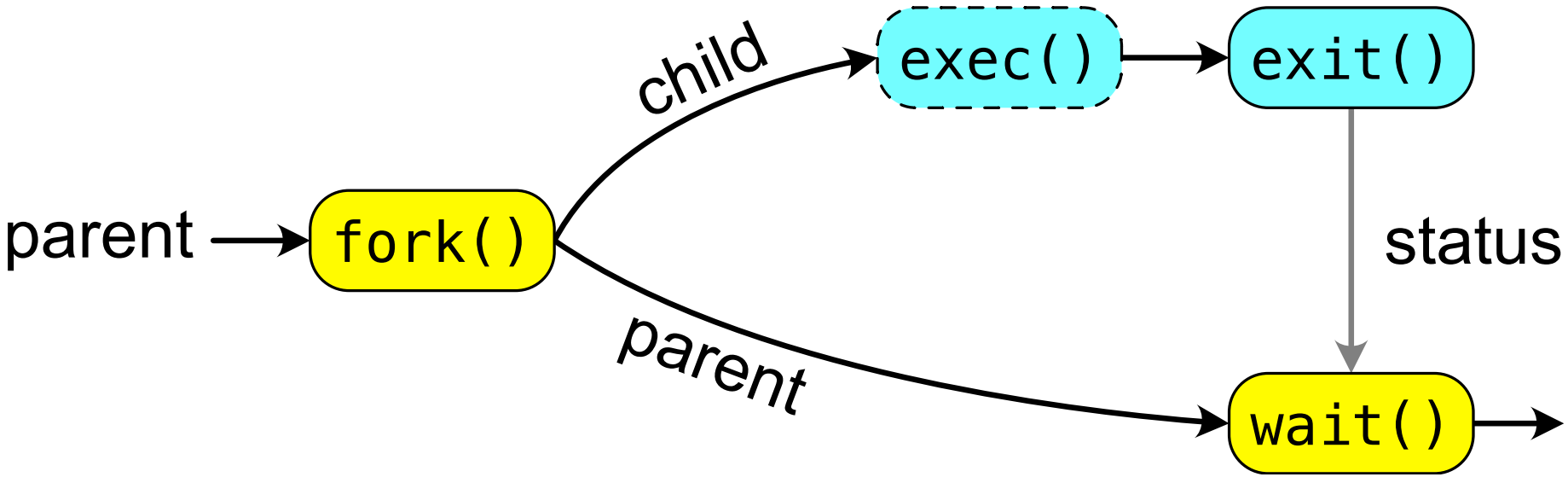
Communication between parent and child processes

Recall the process tree that we saw in Lecture 14:



- A process can create a child process to do a particular task.
- But to do so, it has to be able to communicate with its child.

Depicting creation of a child process using `fork`



Creating child processes using `fork` (maybe `execve`)

In Unix, a child process is created using the `fork` system call. `fork` creates a new process, consisting of a copy of the address space of the parent process.

- Both parent and child processes will be scheduled to run after the `fork` function call completes successfully—but they receive a different value from the `fork` call:
  - the child process is returned the value 0; whereas
  - the parent process is returned the PID of the child.
- The child process then *may* or *may not* execute an `execve` system call, which loads a new program into its memory space (erasing the copy of the parent).
  - Many different wrappers of `execve` are provided by the C library.

Communication between parent and child processes

After creating a child process, the parent can execute `wait`:

- The `wait` system call moves a process off the ready queue until its child process has terminated.
- When the child process terminates, it provides a status value.

Some questions that might be on your mind:

- Why is the parent's address space copied to the child process if `execve` is just going to wipe it all out?
  - It's possible that the child process does not call `execve`;
  - calling `execve` needs an execution context to pass parameters;
  - Copy-On-Write (COW) is used to avoid the expense of copying.
- Why is the parent process given the child process' PID?
  - So parent can track the progress of its child process.

Process termination requires the `exit` system call

When a process has executed its last statement, it executes the `exit` system call, at which point:

- it may return its status to its parent process (who is `wait`ing);
- all its resources (main memory, open files etc.) are deallocated by the operating system.

Processes can die more abruptly, e.g., being terminated by other processes (owned by the same user) using the Unix `kill` signal.

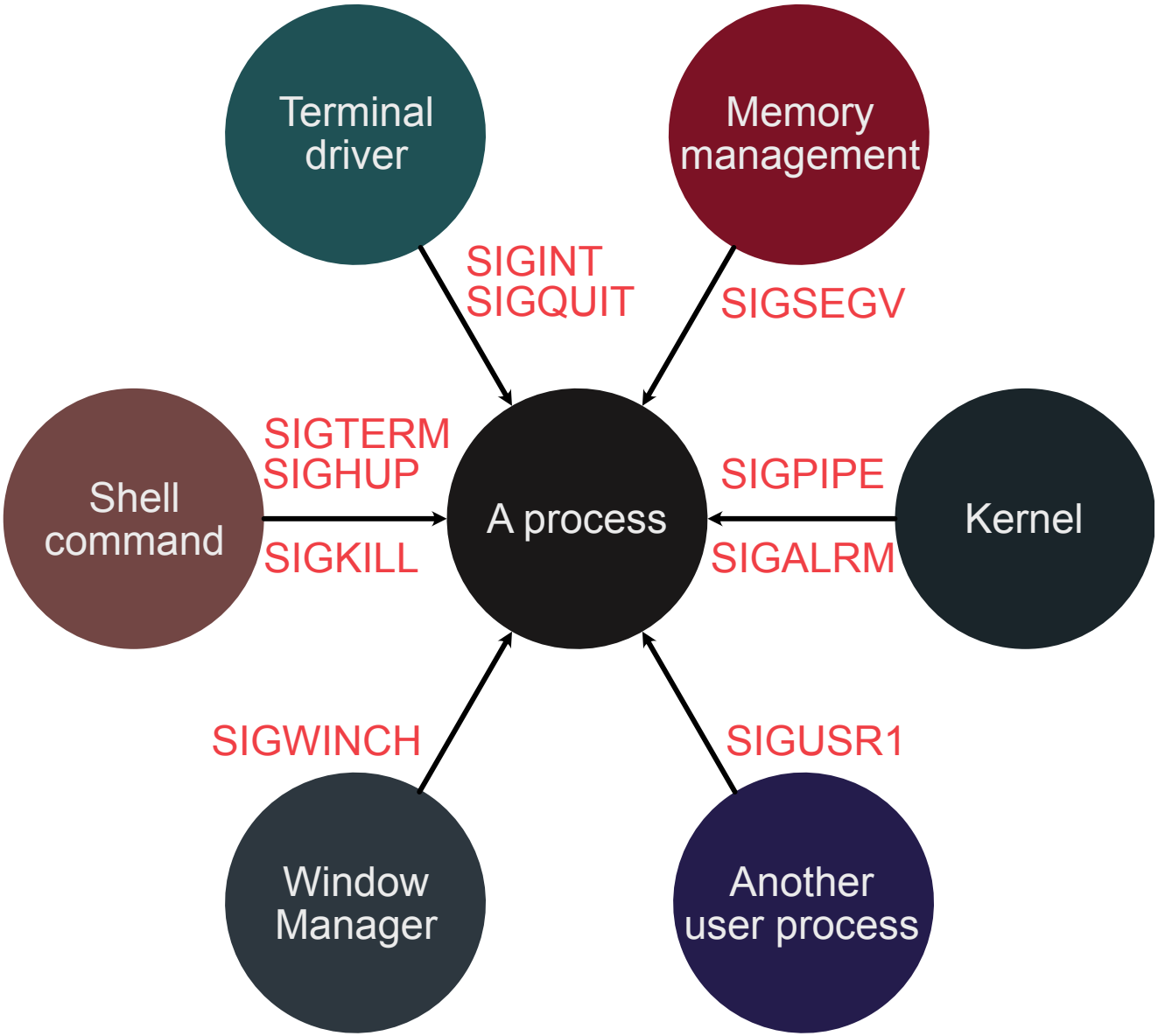
- *Orphan processes*' parent has terminated (get inherited by `init`).
- *Zombie processes* have terminated, but their parent process is not executing `wait`. Since no process is receiving its exit status, it stays in the process table.

Signals—asynchronous inter-process communication

Signals are an important communication mechanism between processes and the OS kernel. For example, a `SIGCHLD` signal is sent to the parent when a child process terminates.

- A process can install a handler (function) to receive a signal—except `SIGKILL`—by using the `signaction` system call.
- The default action for a signal is usually to kill/terminate the process if a handler is not installed.
- Other available actions are ignore, core (dump a large file for debugging), stop (stop the process), and continue. The last two actions are used by debuggers like gdb to debug a program.
- You can run `man 7 signal` on a Linux system to find more information about signals.

Linux signals that a process might expect to receive

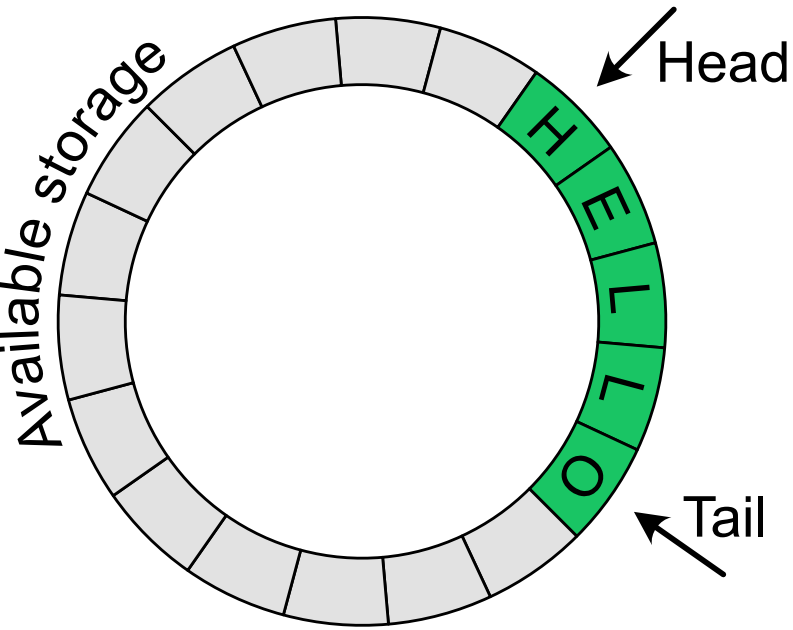


Some common signal types

- `SIGHUP` (1, Term), Hangup detected on controlling terminal or death of controlling process
- `SIGILL` (4, Core), Illegal Instruction
- `SIGABRT` (6, Core), Abort signal from abort(3)
- `SIGFPE` (8, Core), Floating point exception
- `SIGKILL` (9, Term), Kill signal
- `SIGSEGV` (11, Core), Invalid memory reference
- `SIGPIPE` (13, Term), Broken pipe: write to pipe with no readers
- `SIGALRM` (14, Term), Timer signal from alarm(2)
- `SIGTERM` (15, Term), Termination signal (more polite than `KILL`)
- `SIGUSR1` (30,10,16, Term), User-defined signal 1
- `SIGUSR2` (31,12,17, Term), User-defined signal 2
- `SIGCHLD` (20,17,18, Ign), Child stopped or terminated
- `SIGCONT` (19,18,25, Cont), Continue if stopped
- `SIGSTOP` (17,19,23, Stop), Stop process

Pipes: convenient inter-process communication

- Pipes provide a way to establish a communication channel that connects one process' input with another process' output.
- A pipe is represented as "`|`" within a shell command line.
  - For example, `cat myfile | grep keyword`.
- A pipe effects communication between producer/consumer processes using a circular buffer.
- The OS kernel uses signals to notify the producer/consumer processes when the buffer is full/empty.



Readings

For this lecture, the relevant material is in Chapter 3 (Sections 1–3) of Silberschatz *et al.*